1 Documentation of the `deterministic_loss` function

The function `deterministic_loss` computes the total portfolio loss under a single hazard scenario (e.g., a spatial field of seismic intensities over all assets). In terms of inputs:

- $v \in \mathbb{R}^N$ is the exposure vector, where v_i denotes the exposed value (e.g., replacement cost) of asset i.
- $u \in \{0, \ldots, K-1\}^N$ is the typology index vector mapping each asset to one of K vulnerability curves.
- $C \in \mathbb{R}^{K \times M}$ is the vulnerability matrix containing K discrete curves evaluated on M intensity points.
- $x_{\text{grid}} \in \mathbb{R}^M$ is the monotonically increasing intensity grid defining the curve discretization.
- $h \in \mathbb{R}^N$ is the scenario intensity vector, where h_i is the hazard intensity at asset i.

The computational core is a linear interpolation of the vulnerability curve associated with each asset in order to obtain its *Mean Damage Ratio* (MDR), followed by an exposure-weighted aggregation:

$$J = \sum_{i=1}^{N} v_i \, MDR_i.$$

From an implementation standpoint, the routine is designed to be:

- **Vectorized:** avoids Python loops and operates on tensors.
- **Numerically robust:** bounds indices and introduces an ε term in divisions.
- **GPU-friendly:** uses flat indexing for accessing the vulnerability matrix C.

1.1 Graph compilation: `@tf.function`

The decorator `@tf.function` converts the Python function into a TensorFlow execution graph, i.e., a static sequence of TensorFlow operations. In practice, this typically:

- **Reduces Python overhead** for repeated calls (e.g., within optimization or calibration loops).
- **Enables runtime optimizations** (op fusion, scheduling) and efficient execution on accelerators.

1.2 Interval localization and index bounding

Vulnerability curves are discretized over the grid x_{grid}. To interpolate MDR_i at h_i, the algorithm first identifies the interval $[x_j, x_{j+1}]$ that contains h_i.

- `tf.searchsorted(x_grid, h, side='right')`: for each h_i, returns the insertion position in the sorted grid. Subtracting 1 yields an index j such that $x_j \leq h_i < x_{j+1}$.
- `tf.clip_by_value(idx, 0, M-2)`: enforces $j \in [0, M-2]$ to avoid out-of-bounds access when h_i lies outside the grid range (e.g., truncated or extrapolated intensities).

1.3 Value extraction and efficient indexing

Once j is known, two types of lookups are required: (i) the grid values x_j and x_{j+1}, and (ii) the vulnerability values $C[u_i, j]$ and $C[u_i, j+1]$ on the curve selected by typology u_i.

- `tf.gather`: performs vectorized extraction using indices. It is used both for retrieving $x_{\text{lower}} = x_j$ and $x_{\text{upper}} = x_{j+1}$, and for retrieving vulnerability values.
- `tf.reshape(C, [-1])`: flattens $C \in \mathbb{R}^{K \times M}$ into a vector $c_{\text{flat}} \in \mathbb{R}^{KM}$, enabling consistent 1D indexing.
- **Flat indexing:** maps (u_i, j) to a single index via $\text{flat_idx} = u_i \, M + j$ (and similarly $u_i \, M + (j+1)$).

1.4 Interpolation and aggregation

With the grid bounds and vulnerability values in hand, the method computes an interpolation weight α and then the MDR per asset.

- **Computation of α (Eq. 1):**

$$\alpha_i = \frac{h_i - x_j}{x_{j+1} - x_j + 10^{-8}}.$$

  The 10^{-8} term regularizes the denominator when adjacent grid points are extremely close.

- **Damage interpolation (Eq. 2):**

$$MDR_i = (1 - \alpha_i)\, C[u_i, j] + \alpha_i\, C[u_i, j+1].$$

- **Aggregation (Eq. 3):** `tf.reduce_sum(v * mdr)` sums the individual losses $v_i \cdot MDR_i$ to return the scalar J.

1.5 Reference code (`deterministic_loss`)

```
@tf.function

def deterministic_loss(v: tf.Tensor, u: tf.Tensor, C: tf.Tensor,
                     x_grid: tf.Tensor, h: tf.Tensor) -> tf.Tensor:
    """
    Compute deterministic loss for a single hazard scenario (Manuscript Section 2).

    Implements the differentiable interpolation formulation from manuscript equations 1-3:

    - Eq. 1:  = (h - x_j) / (x_{j+1} - x_j)
    - Eq. 2: MDR_i = (1-)C[u_i,j] + C[u_i,j+1]
    - Eq. 3: J =  v_i  MDR_i

    This function uses linear interpolation to calculate Mean Damage Ratios (MDR)
    from vulnerability curves based on intensity values, then aggregates with
    exposure values to compute total loss.

    Parameters
    ----------
    v : tf.Tensor, shape (N,), dtype float32
        Exposure vector containing replacement costs for N assets  ^N
    u : tf.Tensor, shape (N,), dtype int32
        Typology index vector mapping assets to vulnerability curves  ^N
        Values must be in range [0, K-1]
    C : tf.Tensor, shape (K, M), dtype float32
        Vulnerability matrix with K curves and M intensity points  ^(KM)
        C[k,m] represents Mean Damage Ratio for typology k at intensity x[m]
    x_grid : tf.Tensor, shape (M,), dtype float32
        Intensity grid vector defining curve discretization  ^M
        Must be monotonically increasing
    h : tf.Tensor, shape (N,), dtype float32
        Intensity vector for single scenario  ^N
        h[i] is the ground motion intensity at asset i

    Returns
    -------
    J : tf.Tensor, scalar, dtype float32
        Total portfolio loss for this scenario
```

```
    Notes
    -----
    - Uses tf.searchsorted for efficient grid lookup
    - Adds epsilon (1e-8) to denominators for numerical stability
    - Employs flat indexing for GPU compatibility (Metal backend)
    - Fully differentiable w.r.t. all inputs except u (integer indices)

    Mathematical Details
    --------------------
    For each asset i:
    1. Find grid index j such that x[j]  h[i] < x[j+1]
    2. Compute interpolation weight  = (h[i] - x[j]) / (x[j+1] - x[j])
    3. Interpolate damage: MDR[i] = (1-)C[u[i],j] + C[u[i],j+1]
    4. Compute asset loss: L[i] = v[i]  MDR[i]
    5. Aggregate: J =  L[i]

    Examples
    --------
    >>> v = tf.constant([100000., 200000., 300000.], dtype=tf.float32)
    >>> u = tf.constant([0, 1, 0], dtype=tf.int32)
    >>> C = tf.constant([[0.0, 0.5, 1.0], [0.0, 0.3, 0.8]], dtype=tf.float32)
    >>> x_grid = tf.constant([0.0, 0.5, 1.0], dtype=tf.float32)
    >>> h = tf.constant([0.25, 0.75, 0.5], dtype=tf.float32)
    >>> loss = deterministic_loss(v, u, C, x_grid, h)
    >>> print(loss.numpy())
    """

    N = tf.shape(v)[0]

    # Find grid indices: j such that x[j]  h < x[j+1]
    idx = tf.searchsorted(x_grid, h, side='right') - 1
    idx = tf.clip_by_value(idx, 0, tf.shape(x_grid)[0] - 2)

    # Grid boundaries
    x_lower = tf.gather(x_grid, idx)
    x_upper = tf.gather(x_grid, idx + 1)

    # Interpolation weight  (Eq. 1)
    alpha = (h - x_lower) / (x_upper - x_lower + 1e-8)

    # Gather vulnerability values based on typology u
    K = tf.shape(C)[0]
    M = tf.shape(C)[1]

    # Flatten C and compute flat indices
    c_flat = tf.reshape(C, [-1])
    flat_idx_lower = u * M + idx
    flat_idx_upper = u * M + (idx + 1)

    c_lower = tf.gather(c_flat, flat_idx_lower)
    c_upper = tf.gather(c_flat, flat_idx_upper)

    # Mean Damage Ratio (Eq. 2)
    mdr = (1.0 - alpha) * c_lower + alpha * c_upper

    # Total Loss (Eq. 3)
    J = tf.reduce_sum(v * mdr)

    return J
```

1.6 Summary of key TensorFlow operators

Operator	Mathematical / Operational role
`tf.searchsorted`	Interval localization on a sorted grid.
`tf.clip_by_value`	Bounds indices to prevent out-of-range access.
`tf.gather`	Vectorized indexed extraction (gather-style indexing).
`tf.reshape`	Tensor reshaping without changing underlying data.
`tf.reduce_sum`	Reduction operator (summation).

2 Documentation of the `probabilistic_loss_matrix` function

The function `probabilistic_loss_matrix` extends the loss computation to a probabilistic setting, in which Q stochastic event realizations are evaluated simultaneously. The key input is an intensity matrix $H \in \mathbb{R}^{N \times Q}$, where $H[i, q]$ denotes the intensity of event q at asset i.

In terms of inputs/outputs:

- $v \in \mathbb{R}^N$ is the exposure vector (e.g., replacement cost), with v_i associated with asset i.
- $u \in \{0, \ldots, K-1\}^N$ assigns each asset a typology u_i that selects a vulnerability curve.
- $C \in \mathbb{R}^{K \times M}$ is the vulnerability matrix, where $C[k, m]$ is the MDR for typology k at grid point x_m.
- $x_{\text{grid}} \in \mathbb{R}^M$ is the monotonically increasing intensity grid.
- $H \in \mathbb{R}^{N \times Q}$ is the hazard intensity matrix for all assets and events.
- The output is $J \in \mathbb{R}^{N \times Q}$, where $J[i, q]$ is the loss of asset i under event q.

The implementation follows manuscript Eq. 4: for each pair (i, q), the vulnerability curve $C[u_i, \cdot]$ is linearly interpolated at intensity $H[i, q]$ to obtain $MDR[i, q]$, and the loss is obtained by scaling with exposure:

$$J[i, q] = v_i \cdot MDR[i, q].$$

Computationally, the objective is to obtain a routine that is:

- **Fully vectorized** over the NQ combinations, avoiding explicit loops.
- **As memory-efficient as possible,** although the $N \times Q$ output size may dominate.
- **GPU-accelerable,** as the workload is primarily elementwise arithmetic and large-scale `gather` operations.

2.1 Graph compilation: `@tf.function`

The decorator `@tf.function` compiles the computation into a TensorFlow graph. This is particularly relevant here because:

- iterating explicitly over Q events in Python would be prohibitively expensive;
- graph execution enables kernel scheduling and trace reuse across repeated calls.

2.2 Vectorization over $N \times Q$: flattening and reshaping

The algorithm processes the $N \times Q$ matrix problem by temporarily rewriting it as a length-NQ vector problem.

- `tf.reshape(H, [-1])` flattens H into $H_{\text{flat}} \in \mathbb{R}^{NQ}$, enabling fully vectorized grid lookup, indexing, and interpolation.
- After computing $mdr_{\text{flat}} \in \mathbb{R}^{NQ}$, `tf.reshape(mdr_flat, [N, Q])` restores the matrix structure $mdr \in \mathbb{R}^{N \times Q}$.

This "flatten → process → reshape" pattern allows TensorFlow to execute the routine via a small number of dense kernels, exposing substantial hardware parallelism.

2.3 Interval localization and simultaneous interpolation

Given H_{flat}, the computation identifies the grid interval in x_{grid} for each entry.

- `tf.searchsorted(x_grid, H_flat, side='right')` returns insertion positions in the sorted grid; subtracting 1 yields indices j such that $x_j \leq H < x_{j+1}$.
- `tf.clip_by_value(idx, 0, M-2)` ensures indices remain within bounds even if intensities fall outside the grid range.
- `tf.gather(x_grid, idx)` and `tf.gather(x_grid, idx+1)` retrieve the bracketing grid values x_j and x_{j+1}.

The interpolation weight is then computed as

$$\alpha = \frac{H_{\text{flat}} - x_j}{x_{j+1} - x_j + 10^{-8}},$$

a length-NQ vector. The 10^{-8} term increases numerical robustness for small denominators.

2.4 Replicated typologies and flat indexing into C

The vulnerability matrix $C \in \mathbb{R}^{K \times M}$ contains K curves discretized on M intensity points. Each asset i selects a curve via its typology $u_i \in \{0, \ldots, K-1\}$. In the probabilistic setting, u_i does not depend on the event index q, yet the computation must associate each asset with each of its Q intensities $H[i, q]$.

Dimension alignment (replicating typologies).

- `tf.expand_dims(u, 1)` maps u from shape $(N,)$ to $(N, 1)$.
- `tf.tile(u, [1, Q])` replicates this column Q times to produce $u_{\text{rep}} \in \mathbb{Z}^{N \times Q}$.
- `tf.reshape(u_rep, [-1])` yields $u_{\text{flat}} \in \mathbb{Z}^{NQ}$, aligned one-to-one with H_{flat}.

Flat indexing for accessing C. To efficiently retrieve $C[u_i, j]$ and $C[u_i, j+1]$, the implementation uses 1D indexing:

- `tf.reshape(C, [-1])` builds $c_{\text{flat}} \in \mathbb{R}^{KM}$.
- For each component, (u_{flat}, j) is mapped to a linear index

  $$\text{flat_idx} = u_{\text{flat}}\, M + j,$$

  and similarly $u_{\text{flat}}\, M + (j+1)$.
- `tf.gather` then returns the lower and upper vulnerability values in vectorized form.

2.5 Construction of the loss matrix

Given α, $C[u_i, j]$, and $C[u_i, j+1]$ in flattened form, the MDR is computed for each (i, q) pair:

$$MDR_{\text{flat}} = (1-\alpha)\, c_{\text{lower}} + \alpha\, c_{\text{upper}},$$

which is subsequently reshaped to $mdr \in \mathbb{R}^{N \times Q}$.

The loss matrix follows as

$$J[i, q] = v_i \cdot MDR[i, q].$$

In code, `tf.expand_dims(v, 1)` reshapes $v \in \mathbb{R}^N$ into $(N, 1)$ so that multiplication with $mdr \in \mathbb{R}^{N \times Q}$ is performed by broadcasting along the event dimension, without explicitly materializing a length-NQ copy of v.

2.6 Memory considerations

The dominant computational cost of this approach is the materialization of length-NQ intermediate tensors (e.g., H_{flat}, α, indices, and mdr_{flat}) and the output $J \in \mathbb{R}^{N \times Q}$.

Rule-of-thumb estimate (`float32`). Storing the loss matrix alone requires approximately

$$\text{RAM}(J) \approx 4NQ \text{ bytes}$$

when using `float32`. In practice, the *peak* memory during execution is typically several times larger than J because multiple intermediate tensors coexist. As a coarse sizing heuristic, a peak of $\approx 5\times$ the size of J is often reasonable.

N	Q	Elements NQ	J (float32)	Approx. peak ($\approx 5 \times J$)
1,000	1,000	10^6	$\approx$ 4 MB	$\approx$ 20 MB
10,000	10,000	10^8	$\approx$ 0.4 GB	$\approx$ 2.0 GB
10,000	100,000	10^9	$\approx$ 4 GB	$\approx$ 20 GB
50,000	100,000	5×10^9	$\approx$ 20 GB	$\approx$ 100 GB
100,000	100,000	10^{10}	$\approx$ 40 GB	$\approx$ 200 GB

Remark: using `float64` approximately doubles these values. Moreover, on GPU the available device memory is often smaller than system RAM; thus, GPU feasibility is frequently VRAM-limited.

Practical implications.

- For large-scale settings (notably when NQ reaches $\mathcal{O}(10^9)$ or larger), event-wise chunking along Q is recommended, accumulating desired statistics (e.g., expected losses) without retaining the full J matrix in memory.
- It is often effective to store large tensors in `float32` (or lower precision) and reserve `float64` for validation and/or selected aggregations.

CPU vs GPU performance. Performance is mainly governed by NQ and by data-movement costs:

- **CPU:** can be competitive for small-to-moderate sizes, particularly when all tensors reside in system RAM. CPU execution is also robust when the full problem does not fit in GPU memory.
- **GPU:** typically provides clear benefits for large NQ due to high parallelism in elementwise arithmetic and massive gather operations. However, frequent CPU→GPU transfers may dominate runtime and eliminate the speedup.
- **Typical GPU limitation:** device memory. If the workload does not fit, chunking in Q (and, if necessary, also in N) is the standard mitigation.

PC (discrete memory) vs Mac (unified/shared memory). On many PCs with discrete GPUs (e.g., dedicated NVIDIA/AMD devices), CPU tensors reside in system RAM while GPU tensors reside in device VRAM. While drivers may provide virtualization mechanisms, the typical behavior is:

- CPU and GPU allocate from distinct physical memory pools.
- Moving H, C, v, and related tensors to the GPU requires transfers over an interconnect (e.g., PCIe), which incurs non-negligible latency and finite bandwidth.
- VRAM is often substantially smaller than system RAM; consequently, GPU execution becomes memory-limited sooner.

In contrast, on many Macs with Apple Silicon (and, more generally, unified-memory architectures), CPU and GPU can access a shared physical memory pool. This alters the cost profile:

- Explicit RAM→VRAM copies are reduced or avoided because there is no physically separate VRAM.
- Nevertheless, bandwidth is shared: CPU and GPU contend for the same memory subsystem, and large tensor materialization can saturate it.
- The effective limit becomes the available unified memory; large NQ may exhaust system memory and trigger memory compression/paging, severely degrading performance.

Practically, a discrete-GPU PC is most commonly limited by *VRAM exhaustion*, whereas an Apple Silicon Mac is more commonly limited by *overall memory pressure* and shared-bandwidth contention.

On the potential use of `float8`. Reducing numerical precision is a natural avenue for decreasing memory footprint and increasing throughput, yet `float8` requires particular care:

- **Memory reduction:** moving from `float32` (4 bytes) to `float8` (1 byte) would, in principle, reduce the storage cost of J and several intermediates by a factor of approximately four.
- **Numerical risk:** `float8` provides extremely limited dynamic range and mantissa precision; in linear interpolation this may introduce substantial quantization in α and MDR, and may adversely affect gradient-based workflows.
- **Hardware/backend dependence:** any realized speedup depends on efficient `float8` support in the execution backend. Without native support, implicit casts to `float16`/`float32` may eliminate benefits.

A more conservative mixed-precision strategy is often preferable: store H and C in `float16` or `bfloat16` (when supported), while performing aggregations in `float32`. Using `float8` may be viable for inference-like settings, but it requires careful validation of accuracy and numerical stability.

2.7 Reference code (`probabilistic_loss_matrix`)

```
@tf.function
def probabilistic_loss_matrix(v: tf.Tensor, u: tf.Tensor, C: tf.Tensor,
                              x_grid: tf.Tensor, H: tf.Tensor) -> tf.Tensor:
    """
    Compute probabilistic loss matrix for all assets and events (Manuscript Section 3).

    Implements the stochastic loss calculation from manuscript equation 4:
    J[i,q] = v[i]  [(1-[i,q])C[u[i],j[i,q]] + [i,q]C[u[i],j[i,q]+1]]

    This function extends deterministic loss to handle Q stochastic event
    realizations simultaneously, computing the complete NQ loss matrix in a
    single vectorized operation.

    Parameters
```

```
    ----------
    v : tf.Tensor, shape (N,), dtype float32
        Exposure vector for N assets  ^N
    u : tf.Tensor, shape (N,), dtype int32
        Typology index vector  ^N
    C : tf.Tensor, shape (K, M), dtype float32
        Vulnerability matrix  ^(KM)
    x_grid : tf.Tensor, shape (M,), dtype float32
        Intensity grid vector  ^M
    H : tf.Tensor, shape (N, Q), dtype float32
        Hazard intensity matrix  ^(NQ)
        H[i,q] = intensity at asset i during event q

    Returns
    -------
    J_matrix : tf.Tensor, shape (N, Q), dtype float32
        Loss matrix where J[i,q] is the loss of asset i in event q  ^(NQ)

    Notes
    -----
    - Flattens H matrix to (N*Q,) for vectorized interpolation
    - Tiles typology indices Q times to match flattened shape
    - Uses flat indexing for GPU compatibility
    - Reshapes result back to (N, Q) matrix form
    - Fully differentiable w.r.t. v, C, and H

    Memory Considerations
    ---------------------
    For large portfolios, the loss matrix can be substantial:
    - N=10,000 assets, Q=100,000 events  1B elements (4GB float32)
    - Consider chunking over events for very large problems

    Mathematical Details
    --------------------
    The function processes all NQ combinations simultaneously:
    1. Flatten H to vector of length N*Q
    2. Find grid indices for all intensities
    3. Compute interpolation weights  for all combinations
    4. Tile typology vector Q times
    5. Gather vulnerability values using (typology, intensity) pairs
    6. Interpolate MDR for all N*Q combinations
    7. Multiply by exposure (broadcast v across Q events)
    8. Reshape result to (N, Q) matrix

    Examples
    --------
    >>> v = tf.constant([100000., 200000.], dtype=tf.float32)
    >>> u = tf.constant([0, 1], dtype=tf.int32)
    >>> C = tf.constant([[0.0, 0.5, 1.0], [0.0, 0.3, 0.8]], dtype=tf.float32)
    >>> x_grid = tf.constant([0.0, 0.5, 1.0], dtype=tf.float32)
    >>> H = tf.constant([[0.25, 0.75], [0.5, 0.9]], dtype=tf.float32)
    >>> J = probabilistic_loss_matrix(v, u, C, x_grid, H)
    >>> print(J.shape)
    (2, 2)
    """
    N = tf.shape(H)[0]
    Q = tf.shape(H)[1]
    M = tf.shape(x_grid)[0]
```

```
    # Flatten H for vectorized operations
    H_flat = tf.reshape(H, [-1])

    # Find grid indices
    idx = tf.searchsorted(x_grid, H_flat, side='right') - 1
    idx = tf.clip_by_value(idx, 0, M - 2)

    # Grid boundaries
    x_lower = tf.gather(x_grid, idx)
    x_upper = tf.gather(x_grid, idx + 1)

    # Interpolation weight  (broadcast for all NQ combinations)
    alpha = (H_flat - x_lower) / (x_upper - x_lower + 1e-8)

    # Repeat typology indices Q times (for all events)
    u_repeated = tf.tile(tf.expand_dims(u, 1), [1, Q])
    u_flat = tf.reshape(u_repeated, [-1])

    # Use flat indexing for Metal GPU compatibility
    c_flat = tf.reshape(C, [-1])
    flat_idx_lower = u_flat * M + idx
    flat_idx_upper = u_flat * M + (idx + 1)

    c_lower = tf.gather(c_flat, flat_idx_lower)
    c_upper = tf.gather(c_flat, flat_idx_upper)

    # Mean Damage Ratio for all (N, Q) combinations
    mdr_flat = (1.0 - alpha) * c_lower + alpha * c_upper
    mdr_matrix = tf.reshape(mdr_flat, [N, Q])

    # Loss Matrix (Manuscript Eq. 4): J[i,q] = v[i]  MDR[i,q]
    J_matrix = tf.expand_dims(v, 1) * mdr_matrix

    return J_matrix
```

2.8 Summary of key TensorFlow operators

Operator	Mathematical / Operational role
`tf.reshape`	Flattening/reshaping for NQ-level vectorization.
`tf.searchsorted`	Grid-interval localization for all (i, q) pairs.
`tf.clip_by_value`	Bounds indices to $[0, M-2]$.
`tf.tile`	Replicates typologies to align assets with events.
`tf.expand_dims`	Inserts a dimension for tiling and broadcasting.
`tf.gather`	Vectorized retrieval of grid bounds and vulnerability values.